Practical 4: Connecting TikTok Frontend to Backend

Overview

In this practical, we'll connect our Next.js frontend to the Express.js backend that we've built in the previous practicals. We'll focus on implementing user authentication, video display, and social interactions like following users and viewing personalized feeds.

Conceptual Overview

Part 1: API Configuration and Authentication

1. **Setting up API Client**
    - Configure Axios to communicate with our backend
    - Set up request interceptors to attach authentication tokens
    - Handle API error responses

2. **Authentication State Management**
    - Create an authentication context to manage user state
    - Implement JWT-based authentication
    - Store tokens securely in local storage
    - Provide login/logout functionality throughout the application

3. **Authentication User Interface**
    - Create reusable modal components
    - Implement login and registration forms
    - Show appropriate error messages and success notifications

Part 2: Video Feed Integration

1. **Fetching Real Data**
    - Replace mock data with API calls to the backend
    - Handle loading states and errors
    - Format API responses for display

2. **Video Interaction**
    - Implement like/unlike functionality
    - Create comment interface

    - Add proper video playback controls

Part 3: User Discovery and Following

1. **User Search and Discovery**
    - Create an "Explore Users" page
    - Display users with follow/unfollow buttons
    - Show user profiles with videos

2. **Follow/Unfollow Functionality**
    - Implement API calls to follow/unfollow users
    - Update UI to reflect follow status
    - Handle errors and edge cases

3. **Personalized Feed**
    - Create a "Following" feed that shows only videos from followed users
    - Handle empty states when the user isn't following anyone
    - Provide seamless transitions between global and following feeds

Implementation Steps

Refer here for the code updates when following the following steps:
https://github.com/syangche/TikTok_Frontend.git

https://github.com/syangche/TikTok_Server.git

Step 1: API Client Configuration

Provides a standardized way to communicate with the backend API while handling authentication tokens automatically.

First, let's set up our API client to communicate with the backend.

1. Install required packages:

```
npm install axios jwt-decode react-hot-toast
```

2. Create a file at `src/lib/api-config.js`:

   api-config.js: Creates a centralized API client using Axios to make HTTP requests to the backend.

```javascript
import axios from 'axios';

// Define base URL for our API
const API_BASE_URL = 'http://localhost:8000/api'; // Adjust port to yours

// Create main API instance
const api = axios.create({
  baseURL: API_BASE_URL,
  headers: {
    'Content-Type': 'application/json',
  },
});

// Request interceptor for adding authorization
api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('token');
    if (token) {
      config.headers['Authorization'] = `Bearer ${token}`;
    }
    return config;
  },
  (error) => {
    return Promise.reject(error);
  }
);

// Response interceptor for handling errors
api.interceptors.response.use(
  (response) => response,
  (error) => {
    // Handle token expiration
    if (error.response && error.response.status === 401) {
      localStorage.removeItem('token');
      // Optionally redirect to login
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

export default api;
```

3. Create `.env.local` in the root of your project:

```
NEXT_PUBLIC_API_URL=http://localhost:8000/api
```

.env.local: Stores environment variables accessible to the frontend, including the API URL.

Step 2: Authentication Context

Create a context to manage authentication state across the application.

1. Create a file at `src/contexts/authContext.jsx`
2. Update `src/app/layout.js` to include the AuthProvider

authContext.jsx: Creates a React context to manage user authentication state (logged in/out status, user details).

layout.js update: Wraps the application with the AuthProvider so authentication state is available throughout the app.

Step 3: Authentication UI Components

Now let's create the UI components for authentication.

1. Create a modal component at `src/components/ui/Modal.jsx`
2. Create authentication forms at `src/components/auth/AuthForms.jsx`
3. Create an auth modal at `src/components/auth/AuthModal.jsx`

Modal.jsx: Reusable modal component that displays content in a popup window.

AuthForms.jsx: Contains login and signup form components with validation.

AuthModal.jsx: Combines the modal and forms to create a complete authentication UI.

Step 4: Update Main Layout with Authentication

Update your `src/components/layout/MainLayout.jsx` to include login/logout functionality

Note: Updates the layout to conditionally show login/logout buttons and protected navigation items based on authentication status.

Step 5: Create Video Service

Create a service to handle video-related API calls at `src/services/videoService.js`

videoService.js: Contains functions that make API calls related to videos (fetching, liking, commenting). It abstracts video-related API calls into a dedicated service for better organization and reusability.

Step 6: Create User Service

Create a service to handle user-related API calls at `src/services/userService.js`

userService.js: Contains functions for user-related API calls (profile, following, etc.). Handles all user-related data operations by communicating with the backend API.

Step 7: Update VideoCard Component

Update your `src/components/ui/VideoCard.jsx` to handle real data and interactions

VideoCard.jsx: Displays an individual video with user info, video player, and interaction buttons. Updates the component to use real data from the backend and handle real interactions (likes, comments).

Step 8: Update VideoFeed Component

Update your `src/components/ui/VideoFeed.jsx` to fetch real data

VideoFeed.jsx: Displays a feed of videos, either from "For You" or "Following" sources. Fetches and displays videos from the backend with proper loading states and error handling.

Step 9: Create Following Page

Create a new file at `src/app/following/page.jsx`

following/page.jsx: Page that shows videos only from users that the current user follows. Implements a personalized feed based on social connections.

Step 10: Create User Discovery Page

Create a new file at `src/app/explore-users/page.jsx`

explore-users/page.jsx: Page for discovering and following other users on the platform.

Step 11: Create Dynamic Profile Page

Create a new file at `src/app/profile/[userId]/page.jsx`

profile/[userId]/page.jsx: Dynamic page that displays a user's profile based on the userId parameter.

Step 12: Set Up Video Upload

Create or update your video upload page at `src/app/upload/page.jsx`

upload/page.jsx: Page with a form for uploading new videos to the platform. Allows authenticated users to upload videos with captions and thumbnails through a user-friendly interface.

Testing Your Implementation

1. **Register Multiple Users**:
    - Create 2-3 different accounts to test social features

2. **Upload Videos**:
    - Upload videos with different accounts
    - Add captions and thumbnails

3. **Test Following**:
    - Use one account to find and follow other users
    - Verify that followed users appear in the Following feed

4. **Test Video Interaction**:
    - Like/unlike videos
    - View user profiles

    - Test video playback and autoplay

5. **Verify Authentication Flow**:
    - Logout and login with different accounts
    - Test access to protected routes

Resources

- Next.js Documentation
- React Hook Form
- Axios Request Library
- JWT Authentication
- Express.js Documentation
- Prisma ORM Documentation